Bottleneck Analysis Update: GPU vs CPU Greedy Ambiguity Resolution in `traccc`

6344837 – Sorin Bețișor

1. real low-multiplicity ODD events are firmly CPU-favourable,
2. larger synthetic workloads show a clear GPU advantage,
3. the GPU preprocessing phases are already relatively efficient,
4. the main remaining bottleneck is the graph-driven eviction loop,
5. the `n_it` sensitivity sweep suggests that repeated CUDA graph construction dominates more than originally expected.

Experimental scope

The current evidence comes from three complementary benchmark groups:

- a previous 3×3 synthetic benchmark family with candidate counts $n \in \{1000, 5000, 10000\}$ and ambiguity-density presets `low`, `med`, and `high`,
- a real-physics benchmark on 10 ODD `geant4_10muon_1GeV` events,
- an extended GPU scaling sweep plus a fixed-`n_it` sensitivity sweep.

The currently usable real dataset is limited to `geant4_10muon_1GeV`. Other candidate datasets were investigated, but either failed because of geometry mismatches or were not directly usable in the current full-chain setup.

Key quantitative findings

1. Real low-multiplicity physics events

For the 10-event ODD `geant4_10muon_1GeV` benchmark, the mean number of candidates per event is approximately 86.8, with the CPU resolving one event in about 0.370 ms on average. The GPU takes about 2.320 ms on average for the same events, meaning that the GPU is about 6.3× slower in this regime.

Table 1: Real ODD low-multiplicity benchmark summary (10 events).

Metric	CPU	GPU
Mean candidates / event	86.8	86.8
Mean time / event (ms)	0.370	2.320
CPU/GPU ratio	0.16×	
Mean selected tracks	39.7	39.7
Mean removed tracks	47.1	47.1

This is important because it establishes that realistic low-multiplicity ODD events are not currently a good target regime for this GPU resolver.

2. Synthetic crossover behaviour

The extended synthetic sweep shows that the GPU remains slower at small candidate counts but becomes competitive at larger ones. The approximate crossover is between $n \approx 2000$ and $n \approx 3000$ candidates, depending on conflict density. By $n = 5000$, the GPU is already faster in favourable regimes, and by $n = 10000$ the speedup becomes substantial.

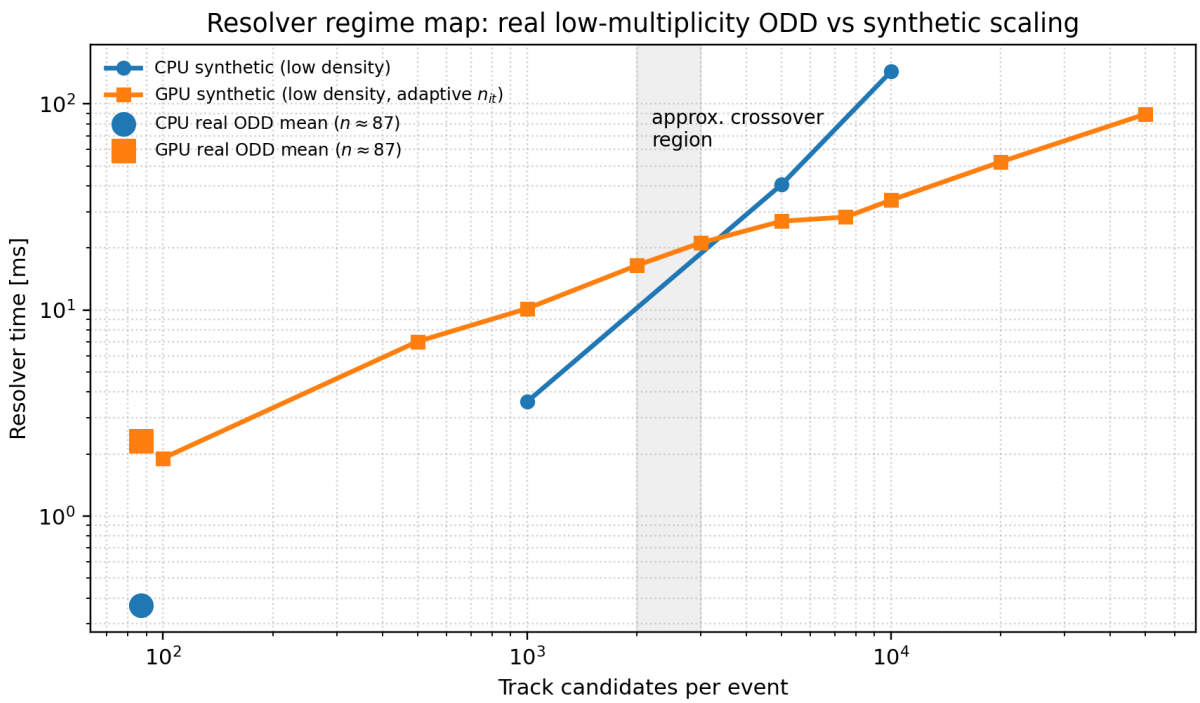

Figure 1: Regime comparison and approximate CPU/GPU crossover. The real ODD point at ~ 87 candidates lies deep in the CPU-favourable region, while larger synthetic workloads show a transition toward GPU advantage.

3. Phase dominance has shifted

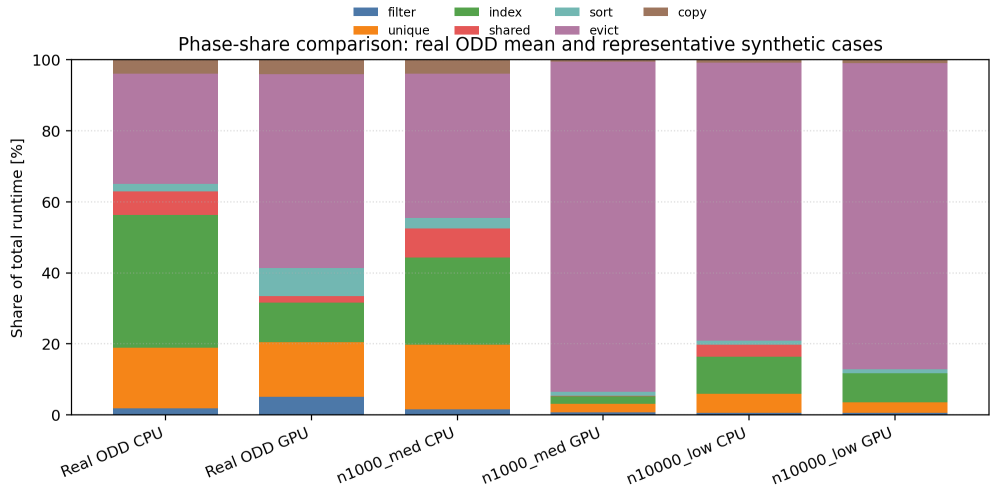

Figure 2: Representative phase-share comparison across regimes. For real low-multiplicity events, preprocessing remains significant; for larger workloads the eviction loop dominates, especially on the GPU.

This supports the interpretation that the next optimization target should not primarily be the already-fast GPU preprocessing kernels, but the iterative eviction mechanism itself.

4. Revised interpretation of the `n_it` issue

Earlier, a natural hypothesis was that the hardcoded `n_it=100` mainly hurts small workloads because it causes over-execution: too many inner graph launches are issued before termination is checked again. The new sensitivity sweep shows that this is not the full story.

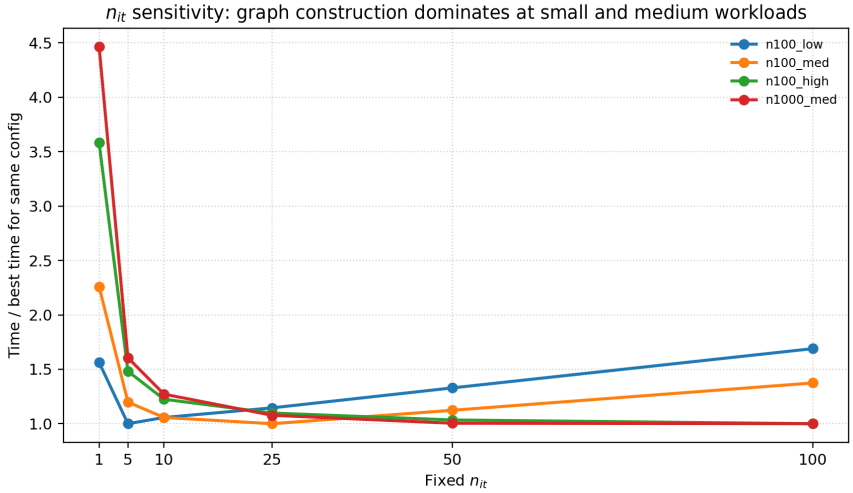

Figure 3: GPU timing sensitivity to fixed `n_it`. The worst case is usually `n_it=1`, while large values often perform best because they amortize graph-construction overhead.

The important result is that `n_it=1` is consistently poor and `n_it=100` is usually near-optimal or best. This is the opposite of the original intuition. The most plausible explanation is that rebuilding the CUDA graph too frequently is more expensive than some wasted inner launches.

5. Graph construction / outer-loop overhead

The sensitivity results motivate a revised bottleneck interpretation: the outer control structure of the GPU loop is a major bottleneck. The pattern is approximately:

```
while (!terminate) {
    // build and instantiate CUDA graph
    for (unsigned int iter = 0; iter < n_it; iter++) {
        cudaGraphLaunch(graphExec, stream);
    }
    // synchronize and read termination state
}
```

Because the graph is built repeatedly in the outer loop, choosing a very small `n_it` causes many graph constructions. That makes the runtime worse even if fewer no-op launches are performed.

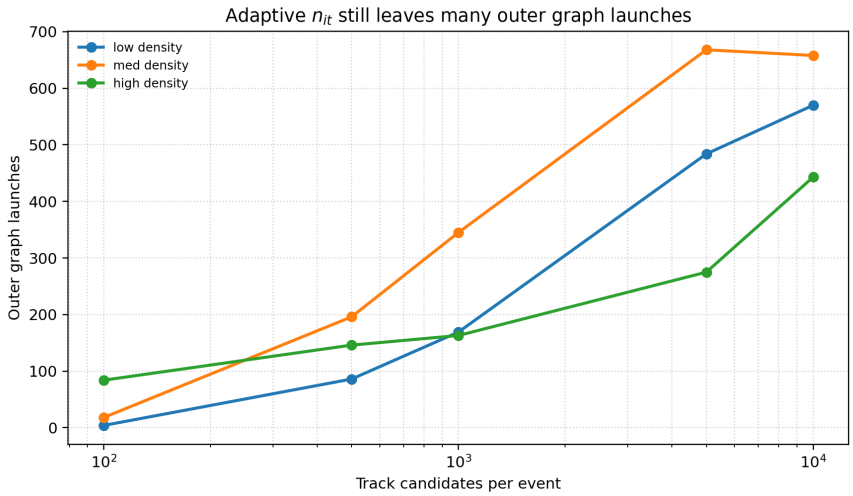

Figure 4: Graph-launch behaviour across representative workloads. The current GPU resolver is highly sensitive to the balance between batched launches and repeated outer-loop graph construction.

Immediate next steps

1. search for a higher-occupancy real dataset or pre-resolver dumps that are compatible with the current setup,
2. investigate whether the graph construction / outer-loop structure can be reduced or reworked,
3. re-run the benchmark suite after any such change to determine whether the crossover point shifts,
4. use the new evidence to refine the final thesis research questions and implementation priorities.